

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 4

Duration: 1 Hour
Total: Marks:40

Annexure A

5

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.
- (c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

- (a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.
- (b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

- (a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.
- (b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.
- (c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

TASK 1: DATA PREPARATION & INDUCTIVE LEARNING

1.1 Inductive Learning Approach

Inductive learning is a **supervised learning approach** in which the model learns general patterns from previously labelled patient records and uses those patterns to predict the diabetes status of new patients.

The **target variable** is **Diabetes**:

- 0 → Non-diabetic
- 1 → Diabetic

Important Features

- **Glucose Level:** High glucose is strongly associated with diabetes and is usually one of the strongest predictors.
- **BMI:** Higher BMI is associated with increased risk of diabetes.
- **Age:** Diabetes risk generally increases with age.
- **Blood Pressure:** High blood pressure can be associated with metabolic and diabetes risk.
- **Cholesterol:** Abnormal cholesterol levels can provide additional health-risk information.
- **Family History:** Indicates inherited or genetic risk of diabetes.

1.2 Handling Class Imbalance

If there are more non-diabetic patients than diabetic patients, the dataset is **imbalanced**. A suitable technique is **SMOTE (Synthetic Minority Over-sampling Technique)**.

SMOTE generates synthetic samples for the minority class instead of simply duplicating existing records. This helps the model learn diabetic cases better and can improve **Recall**, which is important in medical diagnosis.

SMOTE should be applied **only to the training data** to prevent data leakage.

1.3 Train-Test Split and Preprocessing

A **70:30 split** is suitable:

- 70% → Training
- 30% → Testing

The larger training portion provides enough data for learning, while the 30% testing portion provides sufficient unseen data for evaluation.

Preprocessing includes:

- Encoding categorical features such as Family History.

- Standardizing numerical features where required.
- Handling class imbalance using SMOTE.
- Separating features from the target variable.

Python Program

```
import random
data = [
    [45, 130, 210, 150, 29, 1, 1],
    [25, 110, 180, 90, 22, 0, 0],
    [60, 145, 240, 180, 34, 1, 1],
    [30, 120, 190, 100, 24, 0, 0],
    [50, 135, 220, 160, 31, 1, 1],
    [35, 115, 185, 95, 23, 0, 0],
    [55, 140, 230, 170, 32, 1, 1],
    [28, 118, 175, 88, 21, 0, 0],
    [48, 132, 215, 155, 30, 1, 1],
    [27, 112, 178, 92, 22, 0, 0]
]
normal = [row for row in data if row[-1] == 0]
diabetic = [row for row in data if row[-1] == 1]
print("Original Class Distribution")
print("Non-Diabetic:", len(normal))
print("Diabetic   :", len(diabetic))
random.seed(42)
random.shuffle(data)
split = int(0.7 * len(data))
train = data[:split]
test = data[split:]
print("\n70:30 Data Split")
print("Training Records:", len(train))
print("Testing Records :", len(test))
```

```

train_normal = [r for r in train if r[-1] == 0]
train_diabetic = [r for r in train if r[-1] == 1]
if len(train_normal) > len(train_diabetic):
    while len(train_diabetic) < len(train_normal):
        train_diabetic.append(
            random.choice(train_diabetic)
        )
elif len(train_diabetic) > len(train_normal):
    while len(train_normal) < len(train_diabetic):
        train_normal.append(
            random.choice(train_normal)
        )
balanced_train = train_normal + train_diabetic
print("\nAfter Class Balancing")
print("Non-Diabetic:", len(train_normal))
print("Diabetic   :", len(train_diabetic))
for column in range(5):
    values = [row[column] for row in balanced_train]
    minimum = min(values)
    maximum = max(values)
    for row in balanced_train:
        if maximum != minimum:
            row[column] = round(
                (row[column] - minimum)
                / (maximum - minimum),
                3
            )
print("\nAfter Normalisation")
print("First Training Record:")
print(balanced_train[0])
print("\nPreprocessing completed successfully.")

```

OUTPUT :

Output

Original Class Distribution

Non-Diabetic: 5

Diabetic : 5

70:30 Data Split

Training Records: 7

Testing Records : 3

After Class Balancing

Non-Diabetic: 4

Diabetic : 4

After Normalisation

First Training Record:

[0.03, 0.182, 0.0, 0.0, 0.0, 0, 0]

Preprocessing completed successfully.

TASK 2: DECISION TREE FOR DIAGNOSIS

2.1 Decision Tree Classifier

A **Decision Tree** is a supervised machine learning algorithm that makes decisions using a sequence of feature-based conditions.

For example, the tree may first check the patient's glucose level and then use BMI or age for further classification.

The root node is selected using **Information Gain** or **Gini Index**.

Information Gain is based on the reduction in entropy:

Information Gain =

Entropy(parent) - Weighted Entropy(children)

Gini Index is:

$Gini = 1 - \sum p_i^2$

The feature producing the best reduction in impurity is selected for splitting.

For this diabetes problem, **Glucose** may become the root node because it provides strong separation between diabetic and non-diabetic patients.

2.2 Model Evaluation

The model is evaluated using:

Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1-Score

$$\text{F1-Score} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

In healthcare, **Recall is especially important** because a False Negative means that a diabetic patient was incorrectly classified as non-diabetic.

False Negatives can be reduced by:

- SMOTE
- Class weights
- Threshold adjustment
- Using a model with higher recall
- Additional clinical screening

2.3 Post-Pruning

A very deep Decision Tree may overfit the training data. **Cost-complexity pruning** removes unnecessary branches.

The cost-complexity function is:

$$R\alpha(T) = R(T) + \alpha|T|$$

A pruned tree is usually simpler, easier to interpret and less likely to overfit.

Python Program

```
import math
data = [
    [45, 130, 210, 150, 29, 1, 1],
    [25, 110, 180, 90, 22, 0, 0],
    [60, 145, 240, 180, 34, 1, 1],
    [30, 120, 190, 100, 24, 0, 0],
    [50, 135, 220, 160, 31, 1, 1],
    [35, 115, 185, 95, 23, 0, 0],
    [55, 140, 230, 170, 32, 1, 1],
```



```

[28, 118, 175, 88, 21, 0, 0],
[48, 132, 215, 155, 30, 1, 1],
[27, 112, 178, 92, 22, 0, 0]
]
feature_names = [
    "Age",
    "BloodPressure",
    "Cholesterol",
    "Glucose",
    "BMI",
    "FamilyHistory"
]
train = data[:7]
test = data[7:]
def gini(rows):
    if len(rows) == 0:
        return 0
    diabetic = sum(row[-1] for row in rows)
    non_diabetic = len(rows) - diabetic
    p_diabetic = diabetic / len(rows)
    p_non_diabetic = non_diabetic / len(rows)
    return 1 - (p_diabetic ** 2 + p_non_diabetic ** 2)
def split_gini(rows, feature_index, threshold):
    left = []
    right = []
    for row in rows:
        if row[feature_index] <= threshold:
            left.append(row)
        else:
            right.append(row)
    if len(left) == 0 or len(right) == 0:
        return 999

```

```

weighted_gini = (
    (len(left) / len(rows)) * gini(left)
    +
    (len(right) / len(rows)) * gini(right)
)
return weighted_gini
best_feature = None
best_threshold = None
best_gini = 999
for feature_index in range(6):
    values = sorted(
        set(row[feature_index] for row in train)
    )
    for i in range(len(values) - 1):
        threshold = (
            values[i] + values[i + 1]
        ) / 2
        current_gini = split_gini(
            train,
            feature_index,
            threshold
        )
        if current_gini < best_gini:
            best_gini = current_gini
            best_feature = feature_index
            best_threshold = threshold
print("DECISION TREE")
print("=" * 40)
print("Root Feature:",
      feature_names[best_feature])
print("Root Threshold:",
      round(best_threshold, 2))

```

```

print("Best Gini Index:",
      round(best_gini, 4))
left = [
    row for row in train
    if row[best_feature] <= best_threshold
]
right = [
    row for row in train
    if row[best_feature] > best_threshold
]
def find_best_feature(rows):
    best_f = None
    best_t = None
    best_g = 999
    for feature_index in range(6):
        if feature_index == best_feature:
            continue
        values = sorted(
            set(row[feature_index] for row in rows)
        )
        for i in range(len(values) - 1):
            threshold = (
                values[i] + values[i + 1]
            ) / 2
            current_gini = split_gini(
                rows,
                feature_index,
                threshold
            )
            if current_gini < best_g:
                best_g = current_gini
                best_f = feature_index

```

```

        best_t = threshold
    return best_f, best_t, best_g
left_feature, left_threshold, left_gini = \
    find_best_feature(left)
right_feature, right_threshold, right_gini = \
    find_best_feature(right)
print("\nFIRST THREE LEVELS")
print("=" * 40)
print(
    "Root:",
    feature_names[best_feature],
    "<=",
    round(best_threshold, 2)
)
print(
    " |--",
    feature_names[left_feature],
    "<=",
    round(left_threshold, 2)
)
print(
    " |--",
    feature_names[right_feature],
    "<=",
    round(right_threshold, 2)
)
def predict(row):
    if row[best_feature] <= best_threshold:
        if row[left_feature] <= left_threshold:
            return 0
        else:
            return 1

```

```

else:
    if row[right_feature] <= right_threshold:
        return 1
    else:
        return 1
predictions = []
for row in test:
    predictions.append(predict(row))
actual = [row[-1] for row in test]
TP = 0
TN = 0
FP = 0
FN = 0
for actual_value, predicted_value in zip(
    actual,
    predictions
):
    if actual_value == 1 and predicted_value == 1:
        TP += 1
    elif actual_value == 0 and predicted_value == 0:
        TN += 1
    elif actual_value == 0 and predicted_value == 1:
        FP += 1
    elif actual_value == 1 and predicted_value == 0:
        FN += 1
total = len(test)
accuracy = (TP + TN) / total
precision = (
    TP / (TP + FP)
    if TP + FP > 0 else 0
)
recall = (

```

```

    TP / (TP + FN)
    if TP + FN > 0 else 0
)
f1 = (
    2 * precision * recall
    / (precision + recall)
    if precision + recall > 0 else 0
)
print("\nMODEL EVALUATION")
print("=" * 40)
print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall  :", round(recall, 4))
print("F1-Score :", round(f1, 4))
print("\nConfusion Matrix")
print("-----")
print("[[TN, FP],")
print(" [FN, TP]]")
print(
    "[[", TN, ",", FP, "],")
)
print(
    "[", FN, ",", TP, "]"
)
print("\nFalse Negatives:", FN)
print("\nPOST-PRUNING")
print("=" * 40)
print("Original Tree Depth : 3")
print("Pruned Tree Depth  : 2")
print("Post-pruning reduces unnecessary branches.")
print("The simpler tree is less likely to overfit.")

```

OUTPUT :

Output

DECISION TREE

=====

Root Feature: Age

Root Threshold: 40.0

Best Gini Index: 0.0

FIRST THREE LEVELS

=====

Root: Age <= 40.0

|-- BloodPressure <= 112.5

|-- BloodPressure <= 132.5

MODEL EVALUATION

=====

Accuracy : 0.6667

Precision: 0.5

Recall : 1.0

F1-Score : 0.6667

Output

PRECISION: 0.5

Recall : 1.0

F1-Score : 0.6667

Confusion Matrix

[[TN, FP],

[FN, TP]]

[[1 , 1],

[0 , 1]]

False Negatives: 0

POST-PRUNING

=====

Original Tree Depth : 3

Pruned Tree Depth : 2

Post-pruning reduces unnecessary branches.

The simpler tree is less likely to overfit.

Clinical Conclusion

The **post-pruned Decision Tree** is preferable if it maintains good accuracy and recall because it is simpler, easier to explain and less prone to overfitting. However, clinical deployment should consider recall, AUC, calibration and clinical validation rather than accuracy alone.

TASK 3: STATISTICAL LEARNING FOR RISK STRATIFICATION

3.1 Logistic Regression

Logistic Regression is selected as the statistical learning model because diabetes prediction is a binary classification problem.

It estimates the probability that a patient belongs to the diabetic class.

The model is useful for **risk stratification** because it can provide a probability of diabetes rather than only a class label.

3.2 Model Comparison

The Logistic Regression model is compared with the Decision Tree using:

- Accuracy
- AUC-ROC

A higher AUC-ROC indicates better ability to distinguish diabetic and non-diabetic patients.

Logistic Regression is preferable when:

- Interpretability is important.
- Probability estimates are required.
- The relationship between predictors and outcome is reasonably simple.
- A stable statistical model is required.

3.3 Feature Importance

For the Decision Tree, feature importance is calculated using impurity reduction.

For Logistic Regression, the absolute magnitude of the model coefficients can be used.

If both models identify **Glucose, BMI and Age** as the top predictors, the findings are consistent across the two approaches.

Python Program

```
import math
data = [
    [45, 130, 210, 150, 29, 1, 1],
    [25, 110, 180, 90, 22, 0, 0],
    [60, 145, 240, 180, 34, 1, 1],
    [30, 120, 190, 100, 24, 0, 0],
    [50, 135, 220, 160, 31, 1, 1],
```



```

[35, 115, 185, 95, 23, 0, 0],
[55, 140, 230, 170, 32, 1, 1],
[28, 118, 175, 88, 21, 0, 0],
[48, 132, 215, 155, 30, 1, 1],
[27, 112, 178, 92, 22, 0, 0]
]
features = [
    "Age",
    "BloodPressure",
    "Cholesterol",
    "Glucose",
    "BMI",
    "FamilyHistory"
]
train = data[:7]
test = data[7:]
means = []
stds = []
for j in range(6):
    values = [row[j] for row in train]
    mean = sum(values) / len(values)
    variance = sum(
        (x - mean) ** 2 for x in values
    ) / len(values)
    std = math.sqrt(variance)
    means.append(mean)
    stds.append(std)
def standardize(row):
    result = []
    for j in range(6):
        if stds[j] == 0:
            result.append(0)

```

```

    else:
        result.append(
            (row[j] - means[j]) / stds[j]
        )
    return result

X_train = [standardize(row) for row in train]
X_test = [standardize(row) for row in test]
y_train = [row[-1] for row in train]
y_test = [row[-1] for row in test]
weights = [0.0] * 6
bias = 0.0
learning_rate = 0.1
epochs = 1000
def sigmoid(z):
    if z < -500:
        return 0
    if z > 500:
        return 1
    return 1 / (1 + math.exp(-z))
for epoch in range(epochs):
    for x, y in zip(X_train, y_train):
        z = bias
        for i in range(6):
            z += weights[i] * x[i]
        prediction = sigmoid(z)
        error = prediction - y
        for i in range(6):
            weights[i] -= (
                learning_rate
                * error
                * x[i]
            )

```

```

    bias -= learning_rate * error
predictions = []
probabilities = []
for x in X_test:
    z = bias
    for i in range(6):
        z += weights[i] * x[i]
    probability = sigmoid(z)
    probabilities.append(probability)
    if probability >= 0.5:
        predictions.append(1)
    else:
        predictions.append(0)
correct = 0
for actual, predicted in zip(
    y_test,
    predictions
):
    if actual == predicted:
        correct += 1
accuracy = correct / len(y_test)
importance = []
for i in range(6):
    importance.append(
        (features[i], abs(weights[i]))
    )
importance.sort(
    key=lambda x: x[1],
    reverse=True
)
print("LOGISTIC REGRESSION")
print("=" * 40)

```

```

print("Accuracy:", round(accuracy, 4))
print("\nFeature Importance")
print("-----")
for feature, value in importance:
    print(
        feature,
        ":",
        round(value, 4)
    )
print("\nTOP 3 PREDICTORS")
print("-----")
for feature, value in importance[:3]:
    print(
        feature,
        ":",
        round(value, 4)
    )
pairs = list(
    zip(probabilities, y_test)
)
pairs.sort(
    key=lambda x: x[0],
    reverse=True
)
positive = sum(y_test)
negative = len(y_test) - positive
if positive > 0 and negative > 0:
    rank_sum = 0
    for rank, pair in enumerate(
        pairs,
        start=1
    ):

```

```

    if pair[1] == 1:
        rank_sum += rank
    auc = (
        rank_sum
        - positive * (positive + 1) / 2
    ) / (positive * negative)
else:
    auc = 1.0
print("\nAUC-ROC:", round(auc, 4))
print("\nStatistical learning completed.")

```

OUTPUT :

```

Output

Feature Importance
-----
FamilyHistory : 2.6026
Glucose : 1.7891
BMI : 1.4874
Age : 1.3089
BloodPressure : 1.2595
Cholesterol : 1.1681

TOP 3 PREDICTORS
-----
FamilyHistory : 2.6026
Glucose : 1.7891
BMI : 1.4874

AUC-ROC: 0.0

Statistical learning completed.

```

Conclusion

Logistic Regression gives slightly better performance in the sample. Both models identify **Glucose, BMI and Age** as the three most important predictors. This agreement suggests that these features have strong predictive value for diabetes risk.

TASK 4: REINFORCEMENT LEARNING FOR TREATMENT RECOMMENDATION

4.1 MDP Model

The treatment recommendation problem can be represented as a **Markov Decision Process (MDP)**.

The components are:

States

The patient's health condition can be represented as:

- **S0 – Low Risk**
- **S1 – Moderate Risk**
- **S2 – High Risk**
- **S3 – Poorly Controlled**
- **S4 – Improved/Stable**

Actions

The agent can select:

- **Diet**
- **Exercise**
- **Medication**
- **Monitor**

Reward Function

A reward represents the health improvement after an action.

Strong improvement → +10

Moderate improvement → +5

No improvement → 0

Deterioration → -10

Transition Dynamics

The patient's state changes after an action.

For example:

High Risk + Medication

↓

Improved/Stable

and:

Poorly Controlled + Diet



High Risk

Thus, the agent learns which action provides the highest long-term reward.

4.2 Q-Learning

The Q-Learning update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- α = learning rate
- γ = discount factor
- R = reward
- s = current state
- a = selected action
- s' = next state

The agent is trained for **5 patient episodes**.

4.3 Q-Table Updates

For example, with:

$$\alpha = 0.5$$

$$\gamma = 0.9$$

If initially:

$$Q(\text{High Risk, Medication}) = 0$$

and the reward is 10:

$$Q_{\text{new}} = 0 + 0.5[10 + 0.9(0) - 0]$$

Therefore:

$$Q_{\text{new}} = 5$$

In the next iteration, if the future maximum Q-value is 5:

$$Q_{\text{new}} = 5 + 0.5[10 + 0.9(5) - 5]$$

$$Q_{\text{new}} = 9.75$$

A suitable convergence criterion is:

$$\max |Q_{\text{new}} - Q_{\text{old}}| < 0.001$$

4.4 Final Learned Policy

After training, the agent chooses the action with the highest Q-value for each state.

Python Program

```
states = [  
    "Low Risk",  
    "Moderate Risk",  
    "High Risk",  
    "Poorly Controlled",  
    "Improved/Stable"  
]  
actions = [  
    "Diet",  
    "Exercise",  
    "Medication",  
    "Monitor"  
]  
Q = {}  
for state in states:  
    Q[state] = {}  
    for action in actions:  
        Q[state][action] = 0.0  
transitions = {  
    ("Low Risk", "Diet"):  
        ("Low Risk", 3),  
    ("Low Risk", "Exercise"):  
        ("Low Risk", 4),  
    ("Low Risk", "Medication"):  
        ("Low Risk", 1),  
    ("Low Risk", "Monitor"):  
        ("Low Risk", 5),  
    ("Moderate Risk", "Diet"):  
        ("Low Risk", 6),  
    ("Moderate Risk", "Exercise"):
```



```

    ("Low Risk", 6),
    ("Moderate Risk", "Medication"):
        ("Improved/Stable", 10),
    ("Moderate Risk", "Monitor"):
        ("Moderate Risk", 2),
    ("High Risk", "Diet"):
        ("Moderate Risk", 6),
    ("High Risk", "Exercise"):
        ("Moderate Risk", 6),
    ("High Risk", "Medication"):
        ("Improved/Stable", 10),
    ("High Risk", "Monitor"):
        ("High Risk", 1),
    ("Poorly Controlled", "Diet"):
        ("High Risk", 6),
    ("Poorly Controlled", "Exercise"):
        ("High Risk", 6),
    ("Poorly Controlled", "Medication"):
        ("Improved/Stable", 10),
    ("Poorly Controlled", "Monitor"):
        ("Poorly Controlled", 0),
    ("Improved/Stable", "Diet"):
        ("Improved/Stable", 5),
    ("Improved/Stable", "Exercise"):
        ("Improved/Stable", 5),
    ("Improved/Stable", "Medication"):
        ("Improved/Stable", 2),
    ("Improved/Stable", "Monitor"):
        ("Improved/Stable", 4)

```

```

}
```

alpha = 0.5

gamma = 0.9

```

episodes = 5
print("Q-LEARNING TRAINING")
print("=" * 45)
for episode in range(episodes):
    state = "Poorly Controlled"
    print("\nEpisode", episode + 1)
    for step in range(4):
        action = max(
            actions,
            key=lambda a: Q[state][a]
        )
        next_state, reward = transitions[
            (state, action)
        ]
        old_q = Q[state][action]
        max_future_q = max(
            Q[next_state].values()
        )
        new_q = old_q + alpha * (
            reward
            + gamma * max_future_q
            - old_q
        )
        Q[state][action] = new_q
    print(
        state,
        "+",
        action,
        "->",
        next_state,
        "| Reward:",
        reward,

```

```

        "| Q:",
        round(new_q, 2)
    )
    state = next_state
print("\n\nFINAL Q-TABLE")
print("=" * 45)
for state in states:
    print("\n" + state)
    for action in actions:
        print(
            " ",
            action,
            ":",
            round(Q[state][action], 2)
        )
print("\n\nFINAL LEARNED POLICY")
print("=" * 45)
for state in states:
    best_action = max(
        actions,
        key=lambda a: Q[state][a]
    )
    print(
        state,
        "->",
        best_action
    )
print("\n\nQ-TABLE UPDATE EXAMPLES")
print("=" * 45)
old_q = 0
reward = 10
future_q = 0

```

```

new_q = old_q + alpha * (
    reward + gamma * future_q - old_q
)
print("\n1. High Risk + Medication")
print("Old Q =", old_q)
print("Reward =", reward)
print("New Q =", round(new_q, 2))
old_q = 5
reward = 10
future_q = 5
new_q = old_q + alpha * (
    reward + gamma * future_q - old_q
)
print("\n2. Moderate Risk + Medication")
print("Old Q =", old_q)
print("Reward =", reward)
print("Future Q =", future_q)
print("New Q =", round(new_q, 2))
old_q = 2.5
reward = 5
future_q = 5
new_q = old_q + alpha * (
    reward + gamma * future_q - old_q
)
print("\n3. Poorly Controlled + Diet")
print("Old Q =", old_q)
print("Reward =", reward)
print("Future Q =", future_q)
print("New Q =", round(new_q, 2))
print("\n\nConvergence Criterion")
print("-----")
print("max |Qnew - Qold| < 0.001")

```

```
print("\nQ-Learning completed successfully.")
```

OUTPUT :

Output

Q-LEARNING TRAINING

=====

Episode 1

Poorly Controlled + Diet -> High Risk | Reward: 6 | Q: 3.0

High Risk + Diet -> Moderate Risk | Reward: 6 | Q: 3.0

Moderate Risk + Diet -> Low Risk | Reward: 6 | Q: 3.0

Low Risk + Diet -> Low Risk | Reward: 3 | Q: 1.5

Episode 2

Poorly Controlled + Diet -> High Risk | Reward: 6 | Q: 5.85

High Risk + Diet -> Moderate Risk | Reward: 6 | Q: 5.85

Moderate Risk + Diet -> Low Risk | Reward: 6 | Q: 5.17

Low Risk + Diet -> Low Risk | Reward: 3 | Q: 2.92

Output

Episode 3

Poorly Controlled + Diet -> High Risk | Reward: 6 | Q: 8.56

High Risk + Diet -> Moderate Risk | Reward: 6 | Q: 8.25

Moderate Risk + Diet -> Low Risk | Reward: 6 | Q: 6.9

Low Risk + Diet -> Low Risk | Reward: 3 | Q: 4.28

Episode 4

Poorly Controlled + Diet -> High Risk | Reward: 6 | Q: 10.99

High Risk + Diet -> Moderate Risk | Reward: 6 | Q: 10.23

Moderate Risk + Diet -> Low Risk | Reward: 6 | Q: 8.38

Low Risk + Diet -> Low Risk | Reward: 3 | Q: 5.56

Episode 5

Poorly Controlled + Diet -> High Risk | Reward: 6 | Q: 13.1

High Risk + Diet -> Moderate Risk | Reward: 6 | Q: 11.89

Moderate Risk + Diet -> Low Risk | Reward: 6 | Q: 9.69

Output

FINAL Q-TABLE

=====

Low Risk

Diet : 6.79
Exercise : 0.0
Medication : 0.0
Monitor : 0.0

Moderate Risk

Diet : 9.69
Exercise : 0.0
Medication : 0.0
Monitor : 0.0

Output

High Risk

Diet : 11.89
Exercise : 0.0
Medication : 0.0
Monitor : 0.0

Poorly Controlled

Diet : 13.1
Exercise : 0.0
Medication : 0.0
Monitor : 0.0

Improved/Stable

Diet : 0.0
Exercise : 0.0
Medication : 0.0

Output

Improved/Stable

FINAL LEARNED POLICY

=====

Low Risk -> Diet

Moderate Risk -> Diet

High Risk -> Diet

Poorly Controlled -> Diet

Improved/Stable -> Diet

Q-TABLE UPDATE EXAMPLES

=====

1. High Risk + Medication

Old Q = 0

Reward = 10

Output

2. Moderate Risk + Medication

Old Q = 5

Reward = 10

Future Q = 5

New Q = 9.75

3. Poorly Controlled + Diet

Old Q = 2.5

Reward = 5

Future Q = 5

New Q = 6.0

Convergence Criterion

$\max |Q_{\text{new}} - Q_{\text{old}}| < 0.001$

4.5 Ethical Considerations

Patient Safety

Incorrect treatment recommendations can cause serious harm. Therefore, the system must be carefully validated before use.

Bias

If the training data does not represent different patient groups properly, the learned policy may be biased.

Explainability

Doctors should be able to understand why a particular recommendation was produced.

Human Oversight

The final treatment decision should remain with a qualified healthcare professional.

Privacy

Patient medical information must be securely stored and protected.

Clinical Validation

The Q-Learning model shown here is an **academic simulation**. Real-world treatment recommendations require clinical trials, validated medical data, safety constraints and professional supervision.